6COSC022C.2 Advanced Server Side Web Programming

Coursework Report

Name:J.A Nipuna Kaweya Prabashwara

Student ID:20221148/w1953831

Table of Contents

Introduction 3

System Overview 3

High-Level Architecture 4

MVC Architecture 6

Routing and API Design 8

Database Design 9

Load Balancing and Scalability 11

Frontend Architecture 12

Analytics Dashboard Design 13

Data Visualization 15

API Key Scoping & Security 15

Security Architecture 16

Introduction

This project presents the design and implementation of a scalable Alumni Influencer Platform, developed to enhance engagement within an academic community. The system allows alumni to showcase their achievements and compete to become the featured influencer of the day through a structured blind bidding mechanism.

Traditional alumni platforms often lack interactive features and do not provide dynamic visibility for individual achievements. Additionally, there is limited support for structured promotion, sponsorship opportunities, and fair selection mechanisms. To address these challenges, this system introduces a competitive and transparent bidding model, where alumni can place bids using available sponsorship funds. The highest valid bid is selected daily, ensuring fairness while maintaining user engagement.

The platform is built using a hybrid architecture that separates responsibilities across multiple layers. The web application is developed using CodeIgniter 4, which follows the Model-View-Controller (MVC) pattern for handling user interfaces, authentication, and profile management. The business logic and API layer are implemented using Express.js, enabling scalable and modular handling of bidding, sponsorship, and public API services. A MySQL database is used for persistent storage, ensuring data consistency and relational integrity across all system components.

To support scalability and reliability, the system incorporates a load-balanced architecture, allowing multiple instances of both the web and API layers to handle incoming traffic efficiently. Additional features such as API key management, rate limiting, and secure authentication mechanisms are integrated to ensure system security and controlled access.

Overall, the platform demonstrates a modern, modular, and scalable approach to building web applications, combining traditional MVC principles with service-oriented architecture to deliver a flexible and high-performance solution.

System Overview

The Alumni Influencer Platform is designed as an interactive web-based system that enables alumni to actively participate in a competitive and engaging environment. The platform

integrates profile management, sponsorship mechanisms, and a blind bidding system to determine a daily features alumni influencer.

The system provides several key functionalities.First,it support secure user authentication and profile management, allowing alumni to maintain detailed professional profiles that include degrees, certification, licenses, short professional courses, and employment history. These profiles serve as the foundation for showcasing individual achievements.

A central feature of the system is the blind bidding mechanism**,** where users place bids without visibility into competing bids. This ensures fairness and prevents bias in the selection process. The system automatically selects the highest valid bid at the end of each day, designating the winning user as the featured alumni. To maintain fairness, constraints such as monthly participation limits and event-based eligibility are enforced.

The platform also incorporates a sponsorship model**,** where sponsors such as companies, universities, and certification providers can fund alumni activities. These sponsorship offers can be utilized by alumni to place bids, creating a structured and realistic funding ecosystem within the platform.

In addition, the system exposes a set of public API endpoints that allow external clients, such as mobile applications or augmented reality (AR) platforms, to access featured alumni data. This enables integration with external systems and enhances the visibility of alumni achievements beyond the core platform.

Overall, the system combines user interaction, automated decision-making, and external integration to deliver a dynamic and scalable solution that encourages continuous alumni engagement.

High-Level Architecture

The system is designed using a layered approach that separates the web interface, backend services, and data storage. This makes the application easier to manage, scale, and extend in the future.

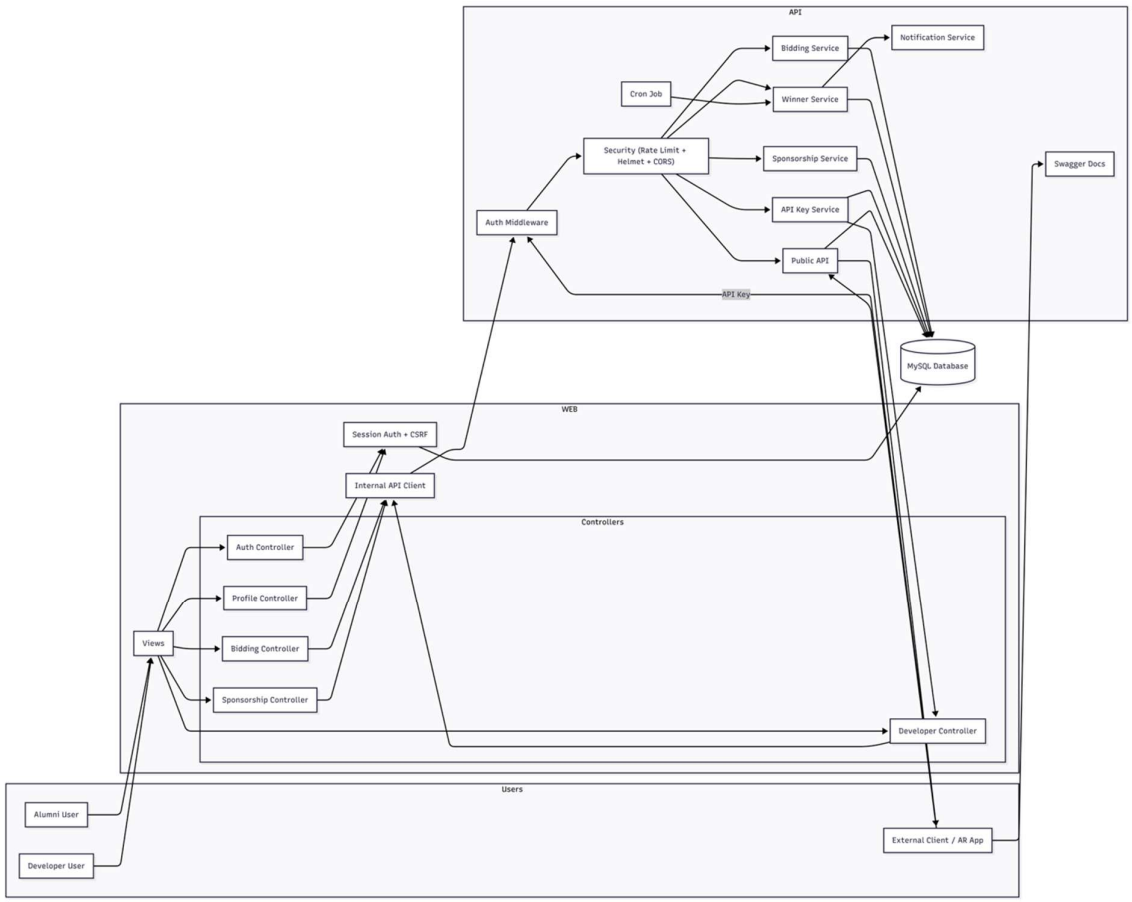

Figure 1:High-Level Architecture Diagram

At the top level, users interact with the system through the web application built using CodeIgniter 4. This layer handles user-facing features such as login, profile management, bidding, and sponsorship interactions. It follows the MVC pattern, where views present data to users and controllers handle incoming requests.

Instead of placing all logic inside the web application, the system uses an internal API client to communicate with a separate backend built using Express.js. This backend is responsible for handling the main business logic, including bidding operations, sponsorship processing, and selecting daily winners. By separating these responsibilities, the system remains clean and easier to maintain.

Before any request reaches the core services, it passes through security components such as authentication middleware and additional protections like rate limiting and CORS. This ensures that only valid and authorized requests are processed.

The API layer is organized into multiple services, each handling a specific function. For example, the bidding service manages user bids, the winner service determines the daily featured alumni, and the sponsorship service handles funding-related operations. A scheduled cron job runs automatically to select the daily winner, and a notification service is used to inform users about important updates.

Both the web and API layers rely on a shared MySQL database to store and retrieve data. This includes user accounts, profiles, bids, sponsorships, and system logs. Keeping a centralized database ensures consistency across the system.

In addition, the system provides public API endpoints that allow external applications, such as mobile or AR platforms, to access featured alumni information. These endpoints are secured using API keys to control access and usage.

Overall, this architecture balances simplicity and scalability by combining a traditional web framework with a separate service-based backend. This makes the system flexible enough to handle growth while keeping the codebase organized and maintainable.

MVC Architecture

The system uses the Model-View-Controller (MVC) pattern in the CodeIgniter 4 web layer to organize the application in a structured way. This helps to separate the user interface, application logic, and data handling, making the system easier to manage and update.

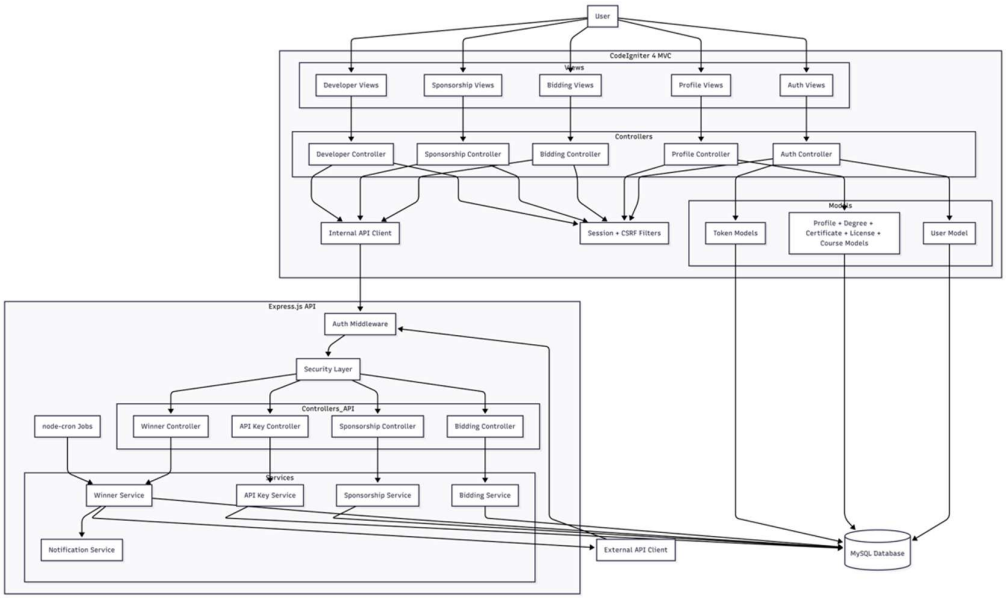

Figure 2:MVC Diagram

At the top level, users interact with the system through different views such as authentication pages, profile pages, bidding pages, sponsorship pages, and developer dashboards. These views are responsible for displaying data and collecting user input, but they do not contain any business logic.

When a user performs an action, the request is handled by the corresponding controller. For example, the Auth Controller manages login and registration, the Profile Controller handles user details, and the Bidding and Sponsorship Controllers manage their respective features. Controllers act as the middle layer that connects the views with the rest of the system.

Inside the web layer, controllers interact with models to retrieve or store data in the database. Models represent the data structure of the system, such as users, profiles, tokens, and other related entities. This keeps all database-related operations organized and separate from the user interface.

In addition to working with models, some controllers also communicate with the backend API through an internal API client. This is used when handling more complex operations such as bidding and sponsorship logic. Instead of placing all logic inside the web layer, these operations are delegated to the API layer, which makes the system more modular and easier to scale.

Security is also integrated within the MVC flow. Session-based authentication and CSRF protection are applied to ensure that only authorized users can access certain features and that requests are safe.

Overall, the MVC structure helps keep the application clean and organized. By separating responsibilities between views, controllers, and models, the system becomes easier to maintain, test, and extend in the future.

Routing and API Design

The system uses a structured routing approach to handle requests from both users and external applications. The routing flow is divided into two main parts: the CodeIgniter web routes and the Express.js API routes.

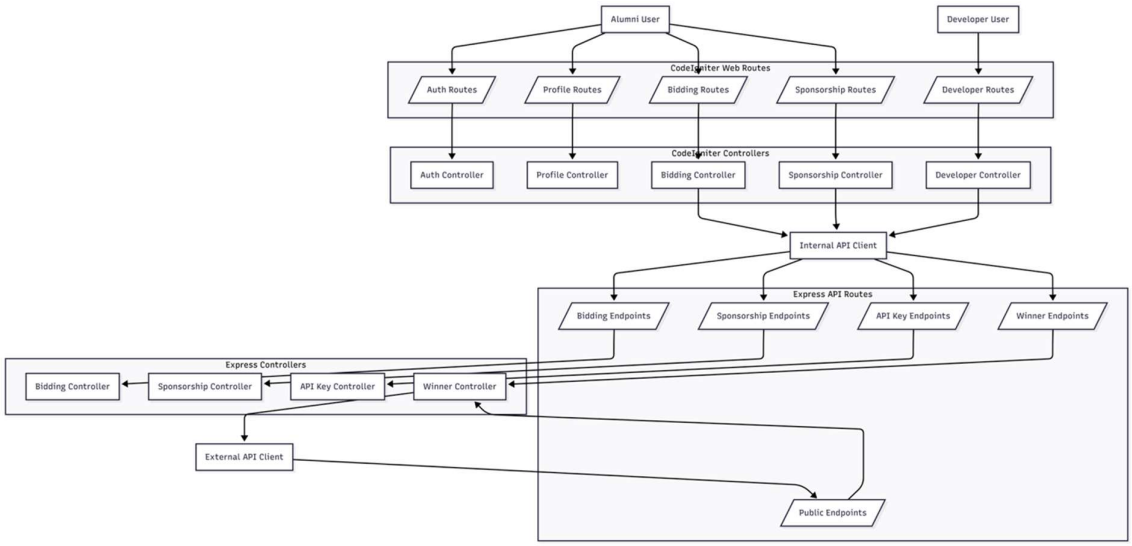

Figure 3:Routing and API Design

On the web side, users interact with different routes such as authentication, profile management, bidding, sponsorship, and developer features. Each of these routes is connected to a specific controller in the CodeIgniter application. For example, authentication routes are handled by the Auth Controller, while bidding-related requests go to the Bidding Controller. This keeps the routing organized and easy to manage.

When simple operations are needed, controllers directly handle the request using models. However, for more complex features like bidding, sponsorship, and API-related actions, the

controllers use an internal API client to send requests to the Express backend. This helps keep the web layer lightweight and focused only on handling user interaction.

On the backend side, the Express.js API is organized into multiple endpoint groups. These include bidding endpoints, sponsorship endpoints, API key endpoints, and winner endpoints. Each group is connected to its own controller, which processes the request and passes it to the relevant service.

The system also supports public endpoints that can be accessed by external clients such as mobile apps or AR applications. These endpoints allow external systems to retrieve data like the featured alumni. Access to these endpoints is controlled using API keys to ensure security.

Overall, the routing design clearly separates responsibilities between the web layer and the API layer. This makes the system easier to maintain, improves scalability, and allows new features to be added without affecting existing functionality.

Database Design

The system uses a relational database designed to organize data in a structured and efficient manner. The database is implemented using MySQL and follows Third Normal Form (3NF) to reduce redundancy and ensure data consistency. By applying 3NF principles, each table stores data related to a single entity, all non-key attributes are fully dependent on the primary key, and transitive dependencies are eliminated. This results in a well-structured schema that improves data integrity and simplifies maintenance.

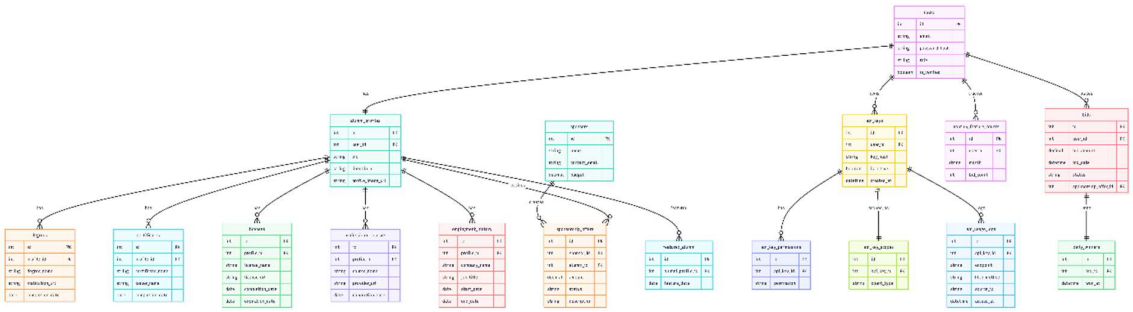

Figure 4:Database ER-Diagram

At the core of the system is the users table, which stores essential authentication and account information such as email, password hash, role, and verification status. Each user is associated with a single record in the alumni profile table, which stores extended profile details including biography, LinkedIn URL, and profile image. This separation ensures that authentication data and profile information are managed independently, aligning with normalization principles.

The alumni profiles table is further connected to multiple related tables that capture professional and academic information. These include degrees, certificates, licenses, professional courses, and employment shistory. Each of these tables is linked using foreign keys, allowing a single profile to have multiple entries in each category. This design avoids duplication of data and ensures that each piece of information is stored only once, which is a key requirement of 3NF.

The system includes a bidding module, which is central to the platform functionality. The bids table stores bid-related information such as bid amount, date, status, and an optional link to a sponsorship offer. Each bid is associated with a user, and winning bids are recorded in the daily winners table. The featured alumni table stores the alumni profiles selected for display on specific dates, enabling the “Alumni of the Day” feature.

To enforce fairness and participation limits, the system uses the monthly feature counts table to track how many times a user has been featured within a given month. This ensures that business rules such as limiting the number of wins per user are properly maintained.

The sponsorship system is implemented using the sponsors and sponsorship offers tables. Sponsors can create funding offers linked to specific alumni profiles, which can then be used when placing bids. This structure maintains clear relationships between sponsors and alumni while avoiding redundant data storage.

For API access and security, the system includes an API key management structure. The API keys table stores API keys linked to users, while API key permissions define the permissions assigned to each key, and API key scopes defines the client type. The API usage logs table records all API requests, including endpoint, method, source IP, and timestamp, enabling monitoring and auditing of API usage.

Overall, the database design adheres to Third Normal Form (3NF), ensuring that data is logically organized, free from redundancy, and consistently maintained. This structure supports

all core system functionalities while providing scalability, reliability, and ease of future extension.

Load Balancing and Scalability

To improve performance and handle multiple users efficiently, a load balancing approach is proposed for this system. Instead of running the application on a single server, the system is designed to distribute incoming traffic across multiple instances. This helps avoid bottlenecks and ensures the system remains stable even when the number of users increases.

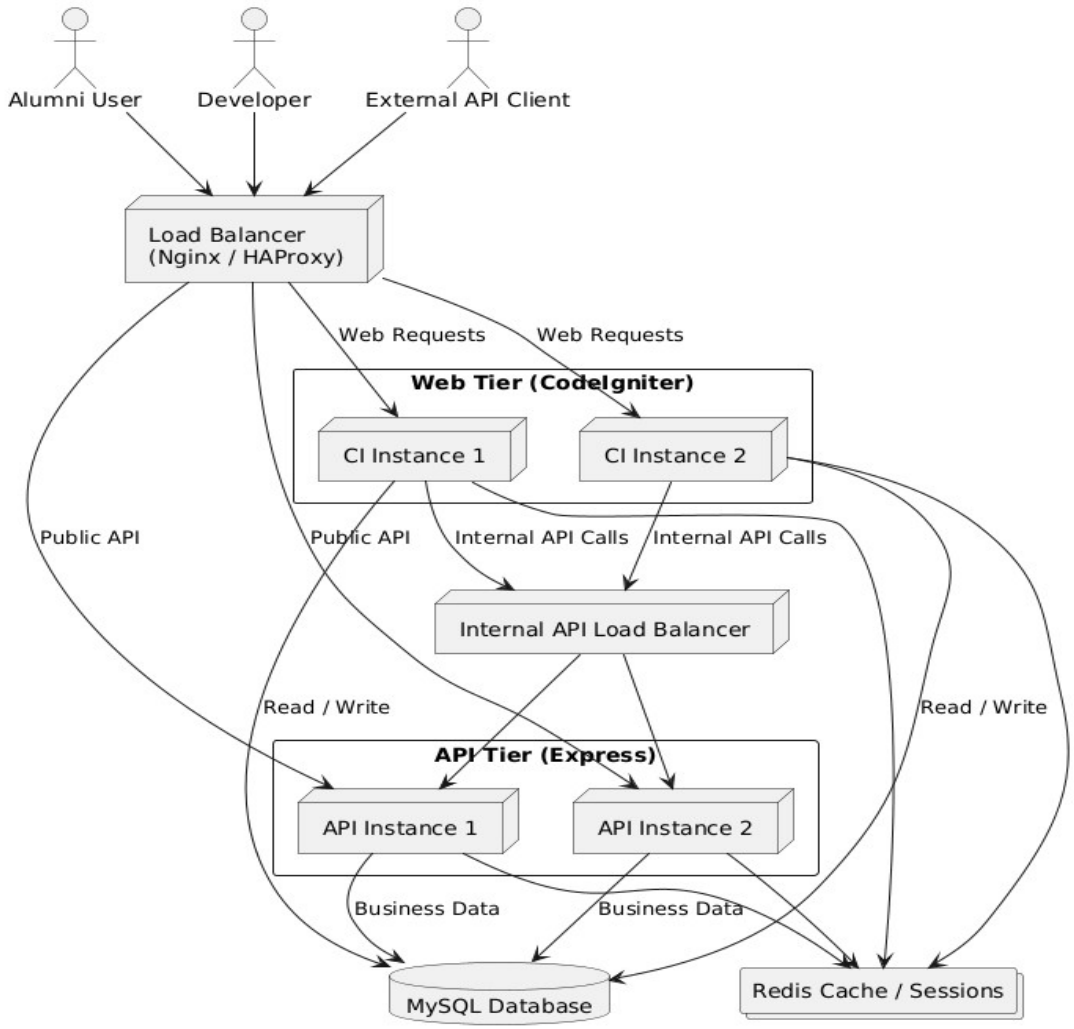

Figure 5:Load Balancing and Scalability Diagram

As shown in the diagram above, all incoming requests from alumni users, developers, and external API clients first go through a load balancer. This load balancer (such as Nginx or

HAProxy) distributes the traffic between multiple CodeIgniter web instances. By doing this, the system can handle more users at the same time without slowing down.

The web layer then communicates with the backend API through an internal API load balancer. This ensures that API requests are also distributed across multiple Express.js instances. As a result, operations like bidding, sponsorship handling, and winner selection can run smoothly without overloading a single server.

Both the web layer and API layer share the same MySQL database, which is used to store all system data. In addition, a Redis cache layer is included to improve performance by handling sessions and frequently accessed data more efficiently.

Overall, this load balancing setup is a proposed design that prepares the system for future growth. It allows the application to scale horizontally, improve reliability, and provide a better experience for users even under high traffic conditions.

Frontend Architecture

The frontend for this project is implemented as a CodeIgniter 4 server-rendered application that functions both as a web-based user interface and as a proxy layer to the Express.js backend API. This design allows the system to maintain secure session-based authentication while efficiently integrating with backend services. All authenticated pages share a common layout template located project Views folder, this ensuring a consistent structure and user experience across the application. The layout is composed of a fixed left sidebar, a top navigation bar, and a main content area. The sidebar, with a fixed width, provides role-based navigation links that dynamically change depending on the logged in user's role, such as admin, alumni, or developer. The top bar displays contextual information including the university name and the currently logged in user's email retrieved from the session. The main content area renders dynamic page content and includes a dedicated flash message zone for displaying system feedback such as success or error notifications. Additionally, a script section is rendered at the bottom of the HTML body, allowing individual pages to inject custom JavaScript code, particularly for Chart.js based data visualization.

The frontend is composed of several reusable components built using Bootstrap 5 and integrated JavaScript libraries. The base layout provides structural elements such as the sidebar and topbar while including essential libraries like Chart.js version 4.4 for data visualization,

jsPDF for report generation, and html2canvas for exporting visual content. Each analytics page includes an inline filter bar that allows users to filter data by programme, graduation year, and industry sector using GET-based form submissions. Key metrics are displayed using stat cards, which are implemented as Bootstrap card components showing an icon, label, and value for quick insights. Data visualizations are presented within chart cards, which are white container elements housing canvas elements used by Chart.js to render interactive charts. Alumni listing pages implement pagination using PHP-generated Bootstrap pagination controls that preserve query parameters across pages. The system also includes a global download functionality that allows users to export charts as images by converting canvas elements into PNG format using data URL (toDataURL) method.

Routing within the frontend is configured in the CodeIgniter routes configuration file and is organized into multiple groups based on user roles. The system enforces access control using CodeIgniter's filter mechanism, ensuring that users can only access permitted for their role. General authenticated routes such as the dashboard and profile management are accessible to all logged in users, while analytics, alumni management, and export functionalities are restricted to administrative users. Similarly, bidding and sponsorship features are limited to alumni users, and developer specific routes are accessible only to developer roles. Access control is implemented through a custom authentication filter that reads the user's role from the session and compares it with the required role defined in the route configuration. If a mismatch occurs, the user is redirected to the dashboard, which then routes them to their appropriate home section using the dashboard controller.

This architecture was chosen due to its balance between simplicity, security and maintainability. By using a server rendered approach the system avoids exposing sensitive authentication tokens on the client side, thereby enhancing security. The use of a unified layout and reusable components improves consistency and reduces code duplication, making the system easier to maintain and extend. Additionally, the integration of frontend rendering with backend API communication allows the application to remain scalable while keeping the frontend logic lightweight. Overall, the design provides an efficient and secure foundation for delivering the analytics dashboard and associated features.

Analytics Dashboard Design

The analytics dashboard is designed as a multi-page interface that allows administrators and other users to explore alumni data through structured views and visual insights. The main

entry point of the system is the dashboard (/dashboard) route, which serves different content depending on the user role. Administrative users are presented with an overview dashboard containing key statistics and mini visualizations, while alumni users are shown a personalized view including their profile summary, bidding information, sponsorship offers, and credential counts. This role-based rendering is handled within the controller by checking the session user role, ensuring that each user only sees relevant information.

For administrative users, the analytics functionality is expanded through a dedicated set of pages under the analytics (/analytics) route. The main analytics page provides an overview consisting of multiple mini charts, giving a quick summary of trends within the alumni dataset. Additional specialized pages are provided for deeper analysis, including skills gap analysis, employment sector distribution, job title trends, top employers, certifications trends, license distribution, a career pathway, and graduation outcomes. Each of these pages focuses on a specific aspect of alumni data and presents it using interactive charts, allowing administrators to gain detailed insights into graduate outcomes and industry trends. The system also includes alumni management pages where administrators can view a list of alumni and inspect individual profiles, as well as export related pages that support report generation.

A key feature of the analytics dashboard is its filtering capability. Each analytics page includes a filter form that allows users to narrow down data based on programme and graduation year. These filter values are retrieved from the backend API and are submitted as query parameters using GET requests. The selected filters persist across page navigation, ensuring consistent user experience. The controller receives these parameters and forwards them to the backend API, which returns filtered data for visualization. This approach ensures that all analytics are dynamic and reflect real-time filtered data rather than static values.

Another important feature is the “Alumni of the Day” section displayed on the administrative dashboard. This feature retrieves data from a public API endpoint and displays the currently featured alumnus, including their profile image, biography, credentials, and LinkedIn link. This provides a real-time highlight of the winning alumnus selected through the bidding system and reinforces the core concept of the platform.

The dashboard also supports multiple export functionalities to enhance usability and reporting. Users can download individual charts as PNG images using a global JavaScript function that captures the chart canvas and converts it into an image format. In addition, a

PDF report generation feature allows all analytics charts to be compiled into a single document. This is implemented by rendering all charts on a dedicated page, capturing the content using html2canvas, and generating a downloadable PDF using jsPDF. Furthermore, the system supports CSV export functionality, where filtered data can be downloaded as a CSV file. This is handled on the server side by streaming data directly to the output using standard file handling functions, ensuring efficient data export.

Overall, the analytics dashboard is designed to provide a comprehensive and user-friendly interface for exploring alumni data. By combining role-based views, dynamic filtering, multiple visualization pages, and export capabilities, the system enables administrators to gain meaningful insights while maintaining flexibility and ease of use.

Data Visualization

All data visualizations in the system are implemented using Chart.js (version 4.4.0), which is loaded through a CDN in the main layout. The chart data is generated from the backend API and passed from PHP to JavaScript as a JSON-encoded variable, allowing dynamic rendering of charts based on real-time data. Different chart types are used depending on the data being represented, such as bar charts for comparisons, line charts for trends, doughnut and polar area charts for distributions, and stacked or grouped bar charts for multi-category analysis. For example, the skills gap chart uses color coding to highlight critical levels, while employment sectors use percentage-based tooltips. Some charts include additional features like toggling between views, hover effects, and smooth animations to improve readability. The main analytics overview page displays all charts together in a grid layout, giving users a quick summary, with options to view each chart in detail on separate pages.

API Key Scoping & Security

The system implements a structured API key scoping mechanism to ensure secure and controlled access for different types of client applications. Three main client types are defined, the analytics dashboard, the AR application, and third-party clients. Each client type is assigned a specific set of permissions based on its intended functionality. For example, the analytics dashboard is allowed broader access with permissions such as read alumni (read:alumni), read analytics (read:analytics), read export (read:export), while the AR and third party clients are restricted to a single permission, read the featured alumini of day (read:alumni_of_day), which limits them to accessing only the featured alumni endpoint. This separation ensures that even

if an API key is compromised, it cannot be used to access unrelated or sensitive endpoints, thereby reducing potential security risks.

The permissions associated with each API key are stored in a dedicated database table, where each key can have multiple permissions but no duplicates, enforced through a unique constraint. In addition, each API key is linked to a single client type through a separate scope table, ensuring a clear mapping between the key and its intended usage. When a developer generates an API key through the developer interface, they first select the client type from a dropdown menu. This request is then sent to the backend, where it is validated and securely forwarded to the API service. The system generates a strong random key using cryptographic methods, prefixes it for identification, and then hashes it using SHA-256 before storing it in the database. Importantly, the raw key is only shown once to the user and is never stored in plain text, which prevents exposure in case of database breach.

To enforce access control, every protected API endpoint is guarded by middleware that checks whether the incoming request has the required permission. When a request is made using an external API key, the system verifies the key and queries the database to confirm that the required permission is assigned. If the permission is missing, the request is denied with a 403 response, and the system provides feedback indicating both the required permission and the permissions currently assigned to the key. This helps developers debug access issues more easily during development. Internal communication between the CodeIgniter frontend and the Express backend is handled differently, using a secure internal secret header. These internal requests bypass permission checks entirely, as they are considered trusted server-to-server communication.

Overall, this API key scoping and security design provides a clear separation of responsibilities between different clients, enforces strict access control, and follows best practices such as hashing sensitive data and limiting permission exposure. This ensures that the system remains secure, scalable, and aligned with the coursework requirements for proper API key management and enforcement.

Security Architecture

The system is designed with multiple layers of security to protect user data and prevent common web-based attacks. Rather than relying on a single mechanism, a combination of security techniques is applied across both the web application and the API layer. This layered

approach improves overall system resilience by reducing vulnerabilities and ensuring that even if one layer is compromised, additional protections remain in place.

At the authentication level, different methods are used depending on how the system is accessed. Users interacting through the web interface are authenticated using secure session-based authentication, while external clients access the system through API keys passed as bearer tokens. Communication between the web application and the backend API is secured using a private internal secret, ensuring that only trusted server-to-server requests are allowed to bypass standard authorization checks.

User credentials are handled securely using strong hashing techniques. During registration, passwords are validated for complexity and then hashed using bcrypt before being stored in the database. During login, the provided password is compared against the stored hash, ensuring that raw passwords are never stored or exposed. This significantly reduces the risk of credential leakage in the event of a data breach.

The system also uses secure token mechanisms for features such as email verification, password reset, and API authentication. Tokens are generated using cryptographically secure random values and are stored in hashed form rather than plain text. They are time-limited and designed for single use, which helps prevent replay attacks and unauthorized reuse.

API key management is implemented with a strict scoping mechanism to control access for different types of clients. Each API key is associated with a specific client type and assigned a predefined set of permissions. These permissions define exactly which endpoints the key is allowed to access. For example, some clients may only retrieve limited public data, while others can access broader analytics functionality. All API keys are securely generated using cryptographic methods, prefixed for identification, and hashed using SHA-256 before being stored. The original key is only displayed once at creation time and is never stored in plain text.

Access control is enforced through middleware that validates every incoming request. For external API requests, the system verifies the bearer token, retrieves the associated permissions, and checks whether the requested operation is allowed. If the required permission is not present, the request is denied with a clear error response. Internal requests between trusted system components are authenticated using a secure internal secret and are allowed to bypass standard permission checks, ensuring efficient and secure internal communication.

Input validation and sanitization are applied throughout the system to prevent malicious data from being processed. All incoming data, including user inputs and API parameters, are validated for correct format and type. For example, numeric values must fall within acceptable ranges, and identifiers must be valid integers. This helps prevent common vulnerabilities such as SQL injection, cross-site scripting, and malformed requests.

Additional security protections include the use of HTTP security headers to mitigate risks such as cross-site scripting and clickjacking. CSRF protection is applied to all form submissions within the web application to ensure that requests originate from trusted sources. On the API side, rate limiting is implemented to control the number of requests a client can make within a given time frame, reducing the risk of abuse, brute-force attacks, and denial-of-service attempts.

Role-based access control is used to ensure that users can only access features appropriate to their role. Similarly, API-level access is restricted based on assigned permissions, ensuring that different clients have clearly defined and limited capabilities. Certain endpoints are reserved for internal use only, while others are exposed externally with strict authorization requirements.

Finally, the system includes logging and monitoring mechanisms to track activity and detect unusual behavior. API usage is recorded, including timestamps and accessed endpoints, allowing administrators to analyze system activity and identify potential threats. These logs also support debugging and help maintain system reliability.

Overall, security architecture combines authentication, authorization, encryption, validation, and monitoring into a comprehensive framework that protects the system from a wide range of threats while ensuring secure and controlled access for all users and clients.

Conclusion

The system presented in this project demonstrates a comprehensive and well-structured approach to developing a modern, scalable web application for alumni engagement and analytics. By integrating a CodeIgniter-based frontend with an Express.js backend, the architecture effectively separates concerns while maintaining flexibility and performance. The use of a relational database designed in Third Normal Form (3NF) ensures data consistency and efficient management of complex relationships. Key features such as the blind bidding system, sponsorship model, and “Alumni of the Day” functionality provide meaningful user interaction and real-world relevance.

The analytics dashboard enhances the system by transforming raw alumni data into valuable insights through dynamic filtering, multiple visualization techniques, and export capabilities. The implementation of Chart.js for interactive charts, along with CSV and PDF export features, enables users to analyze and utilize data effectively. In addition, the system incorporates a robust security architecture, including session-based authentication, API key scoping, permission-based access control, secure token handling, and input validation, ensuring that both internal and external interactions remain secure.

Overall, the system successfully meets its functional and non-functional requirements by combining usability, scalability, and security. The modular design allows for future enhancements, such as advanced analytics, machine learning-based predictions, and integration with additional external platforms. This project highlights the importance of structured architecture, secure design practices, and effective data visualization in the development of real-world web applications.